Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 7

Construcción de variables rezagadas e indicadores simples

Objetivo de la semana. A partir de la base con rendimientos y volatilidad histórica construida en la Semana 6, crear variables predictoras mediante rezagos, medias móviles e indicadores financieros simples. Todas las variables deberán utilizar exclusivamente información disponible hasta la fecha en que son calculadas. Se enfatizará la interpretación temporal de cada predictor, la prevención de fuga de información y la validación manual de las transformaciones realizadas.

Índice

1. Contexto de la Semana 7 **2**

2. Objetivos de aprendizaje **2**

3. La idea de una variable predictora **3**

4. Variables rezagadas **4**

5. Interpretación de `shift()` en pandas **4**

6. Fuga temporal de información **5**

7. Lectura de la base de volatilidad **5**

8. Validaciones de entrada **6**

9. Rezagos de rendimientos **7**

10. Rezagos de magnitud del rendimiento **8**

11. Rezagos de rendimientos al cuadrado **8**

12. Rezagos de volatilidad histórica **9**

13. Verificación manual de un rezago **9**

14. Valores faltantes generados por los rezagos **10**

15. Medias móviles de los rendimientos **11**

16. Verificación manual de una media móvil **11**

17. Media móvil del rendimiento absoluto **12**

18. Comparación entre volatilidad y movimiento absoluto promedio **12**

19. Indicador de momentum **13**

20. Verificación del momentum **14**

21. Cambio relativo en el volumen **15**

22. Rango intradiario **15**

23. Relación entre volatilidad de corto y largo plazo **16**

24. Visualización del cociente de volatilidades **17**

25. Resumen de variables construidas **17**

26. Inspección de las nuevas variables **18**

27. Estadística descriptiva **19**

28. Matriz de correlación exploratoria **19**

29. Valores faltantes en la base de predictores **20**

30. Validaciones automáticas adicionales **20**

30.1. Rezagos correctos 20

30.2. Rango intradiario no negativo 21

30.3. Cociente de volatilidades finito 21

30.4. No utilizar desplazamientos futuros 21

31. ¿Debe estandarizarse la base en esta semana? **21**

32. Construcción de la base final de la semana **22**

33. Exportación **22**

34. Verificación del archivo exportado **23**

35. Ejercicios **23**

36. Checklist de cierre **24**

37. Producto esperado **25**

1. Contexto de la Semana 7

En la Semana 6 se construyeron medidas de volatilidad histórica utilizando ventanas móviles de 5, 10, 20 y 30 días. El archivo de entrada para esta semana es

`data/processed/SPY_2015_2025_volatility.csv`.

La base contiene, entre otras variables,

`Return`, `Abs_Return`, `Sq_Return`,

así como

`Vol_5`, `Vol_10`, `Vol_20`, `Vol_30`.

Hasta este momento las variables construidas describen el comportamiento observado del mercado. El siguiente paso consiste en organizar esa información histórica en variables que posteriormente puedan utilizarse como entradas de modelos predictivos.

La secuencia conceptual de esta semana será

información histórica $\longrightarrow$ rezagos e indicadores $\longrightarrow$ variables predictoras.

Delimitación de la semana. En esta etapa no se construirá todavía la volatilidad futura que funcionará como variable objetivo. Tampoco se ajustarán modelos predictivos. La Semana 7 se concentra exclusivamente en la construcción y validación de variables explicativas.

2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. explicar qué es una variable rezagada;
2. interpretar correctamente el operador temporal `shift()`;
3. construir rezagos de rendimientos, rendimientos absolutos y rendimientos al cuadrado;
4. construir rezagos de medidas de volatilidad histórica;
5. calcular medias móviles de rendimientos;
6. calcular medias móviles de rendimientos absolutos;

7. construir indicadores sencillos de momentum;
8. calcular cambios relativos en el volumen de operación;
9. construir una medida simple del rango intradiario;
10. interpretar el cociente entre volatilidad de corto y largo plazo;
11. identificar valores faltantes inducidos por rezagos y ventanas móviles;
12. verificar manualmente que una variable rezagada contiene la observación correcta;
13. distinguir una variable explicativa presente de una variable objetivo futura;
14. identificar situaciones de fuga temporal de información;
15. exportar una base de variables predictoras que sirva como entrada para la Semana 8.

3. La idea de una variable predictora

Supongamos que deseamos realizar una predicción utilizando toda la información disponible hasta una fecha t. En ese momento se conocen, por ejemplo,

$$P_t, \quad r_t, \quad |r_t|, \quad r_t^2,$$

así como las observaciones anteriores

$$r_{t-1}, \quad r_{t-2}, \quad r_{t-3}, \ldots$$

y medidas retrospectivas como

$$\sigma_{t,5}, \quad \sigma_{t,10}, \quad \sigma_{t,20}, \quad \sigma_{t,30}.$$

Todas estas cantidades pueden utilizarse para construir predictores.

Por el contrario, en el tiempo t no conocemos

$$r_{t+1}, \quad r_{t+2}, \ldots$$

ni ninguna variable que dependa de esas observaciones futuras.

> **Principio fundamental.** Una variable predictora asociada a la fecha t puede depender de información correspondiente a t o a fechas anteriores, pero nunca de observaciones posteriores a t.

4. Variables rezagadas

Sea una serie temporal x_t. El rezago de orden k se define como

$$L^k x_t = x_{t-k}.$$

Por ejemplo,

$$Lr_t = r_{t-1}, \qquad L^2 r_t = r_{t-2}, \qquad L^5 r_t = r_{t-5}.$$

Estas variables permiten que un modelo posterior utilice información sobre el comportamiento reciente de la serie.

Para los rendimientos se construirán los rezagos

$$r_{t-1}, \qquad r_{t-2}, \qquad r_{t-5}, \qquad r_{t-10}.$$

En la base se almacenarán como

`Return_Lag_1`, `Return_Lag_2`, `Return_Lag_5`, `Return_Lag_10`.

5. Interpretación de `shift()` en pandas

En `pandas`, una variable rezagada puede construirse mediante

```
datos["Return_Lag_1"] = datos["Return"].shift(1)
```

El valor almacenado en una fecha t es entonces el rendimiento correspondiente a la fecha anterior.

Es decir,

$$\texttt{Return_Lag_1}_t = r_{t-1}.$$

De manera análoga,

```
datos["Return_Lag_5"] = datos["Return"].shift(5)
```

produce

$$\texttt{Return_Lag_5}_t = r_{t-5}.$$

Para observarlo con claridad, conviene comparar las primeras filas:

```
datos[
    [
        "Return",
        "Return_Lag_1",
        "Return_Lag_2",
        "Return_Lag_5"
    ]
].head(12)
```

6. Fuga temporal de información

Uno de los errores más importantes en problemas de predicción temporal consiste en utilizar accidentalmente información futura.

Por ejemplo,

```
datos["Ejemplo_incorrecto"] = (
    datos["Return"].shift(-1)
)
```

produciría

$$\texttt{Ejemplo_incorrecto}_t = r_{t+1}.$$

En consecuencia, esta columna contiene información que todavía no estaba disponible en t.

> **Regla de la Semana 7.** No se utilizará `shift()` con valores negativos para construir variables explicativas. Los desplazamientos hacia el futuro se reservarán para la construcción formal de la variable objetivo en la Semana 8.

7. Lectura de la base de volatilidad

El notebook de esta semana puede llamarse

`07_variables_predictoras.ipynb`.

Se utilizarán las bibliotecas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

Las rutas principales serán

```
ruta_processed = Path("data/processed")

archivo_volatility = (
    ruta_processed /
    "SPY_2015_2025_volatility.csv"
)

archivo_features = (
    ruta_processed /
    "SPY_2015_2025_features.csv"
)
```

Antes de leer la base:

```
if not archivo_volatility.exists():
    raise FileNotFoundError(
        f"No se encontro el archivo: {archivo_volatility}"
    )
```

Después:

```
datos_vol = pd.read_csv(
    archivo_volatility,
    index_col=0,
    parse_dates=True
)

datos = datos_vol.copy(deep=True)
```

Conviene inspeccionar la estructura mediante

```
print(datos.shape)
print(datos.index.min())
print(datos.index.max())
print(datos.columns.tolist())

datos.head()
```

8. Validaciones de entrada

La Semana 7 depende directamente de la base construida en la semana anterior. Por ello se verificarán las columnas necesarias:

```
columnas_necesarias = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
```

```
    "Volume",
    "Return",
    "Abs_Return",
    "Sq_Return",
    "Vol_5",
    "Vol_10",
    "Vol_20",
    "Vol_30"
]

faltantes = [
    col for col in columnas_necesarias
    if col not in datos.columns
]

assert len(faltantes) == 0, (
    f"Faltan columnas: {faltantes}"
)
```

También se verifica el orden temporal:

```
assert datos.index.is_monotonic_increasing
assert datos.index.is_unique
```

Y la integridad de los rendimientos:

```
assert not datos["Return"].isna().any()
assert not datos["Abs_Return"].isna().any()
assert not datos["Sq_Return"].isna().any()

assert np.isfinite(datos["Return"]).all()
```

Las volatilidades sí contienen valores faltantes iniciales, porque fueron construidas mediante ventanas móviles. Esto es esperado.

9. Rezagos de rendimientos

Se utilizarán los rezagos

$$k \in \{1, 2, 5, 10\}.$$

En Python:

```
lags = [1, 2, 5, 10]

for lag in lags:
    datos[f"Return_Lag_{lag}"] = (
        datos["Return"].shift(lag)
    )
```

Estas cuatro variables describen rendimientos observados a diferentes distancias temporales respecto a la fecha actual.

Por ejemplo,

$$\texttt{Return_Lag_10}_t = r_{t-10}.$$

10. Rezagos de magnitud del rendimiento

Además del signo del rendimiento, interesa conservar información sobre la magnitud de movimientos anteriores.

Se construirán rezagos de

$$|r_t|$$

mediante

```
for lag in lags:
    datos[f"Abs_Return_Lag_{lag}"] = (
        datos["Abs_Return"].shift(lag)
    )
```

Por ejemplo,

$$\texttt{Abs_Return_Lag_5}_t = |r_{t-5}|.$$

Estas variables pueden resultar útiles para describir persistencia en la intensidad de movimientos recientes.

11. Rezagos de rendimientos al cuadrado

También se conservarán rezagos de

$$r_t^2.$$

En Python:

```
for lag in lags:
    datos[f"Sq_Return_Lag_{lag}"] = (
        datos["Sq_Return"].shift(lag)
    )
```

Así,

$$\texttt{Sq_Return_Lag_2}_t = r_{t-2}^2.$$

Las variables $|r_t|$ y r_t^2 son dos formas distintas de representar la magnitud de los rendimientos.

12. Rezagos de volatilidad histórica

También es natural utilizar como predictores medidas de volatilidad construidas en la Semana 6.

Para cada ventana

$$w \in \{5, 10, 20, 30\},$$

se construirá un rezago de un día:

```
ventanas_vol = [5, 10, 20, 30]

for w in ventanas_vol:
    datos[f"Vol_{w}_Lag_1"] = (
        datos[f"Vol_{w}"].shift(1)
    )
```

Por ejemplo,

$$\texttt{Vol_20_Lag_1}_t = \sigma_{t-1,20}.$$

Esta variable representa la volatilidad histórica de 20 días que ya estaba disponible al finalizar la sesión anterior.

> **Observación.** Las volatilidades sin rezago, como $\sigma_{t,20}$, también utilizan únicamente información hasta t. Sin embargo, mantener algunas versiones rezagadas permite estudiar posteriormente qué tan útil resulta la persistencia de la volatilidad.

13. Verificación manual de un rezago

Es importante comprobar que `shift()` realiza exactamente el desplazamiento esperado.

Seleccionemos una posición intermedia:

```
i = 100

actual = datos["Return_Lag_1"].iloc[i]
esperado = datos["Return"].iloc[i - 1]
```

```
5
6  print("Fecha actual:", datos.index[i])
7  print("Return_Lag_1:", actual)
8  print("Return anterior:", esperado)
9
10 assert np.isclose(actual, esperado)
```

Para cinco días:

```
1 actual = datos["Return_Lag_5"].iloc[i]
2 esperado = datos["Return"].iloc[i - 5]
3
4 assert np.isclose(actual, esperado)
```

Esta validación convierte la interpretación temporal de los rezagos en una comprobación automática.

14. Valores faltantes generados por los rezagos

Una variable desplazada k posiciones produce k valores faltantes al comienzo de la serie.

Por tanto,

$$\begin{aligned} \texttt{Return_Lag_1}: &\quad 1, \\ \texttt{Return_Lag_2}: &\quad 2, \\ \texttt{Return_Lag_5}: &\quad 5, \\ \texttt{Return_Lag_10}: &\quad 10. \end{aligned}$$

Podemos comprobarlo con

```
1 for lag in lags:
2     variable = f"Return_Lag_{lag}"
3
4     print(
5         variable,
6         datos[variable].isna().sum()
7     )
```

Y verificar automáticamente:

```
1 for lag in lags:
2     variable = f"Return_Lag_{lag}"
3
4     assert (
5         datos[variable].isna().sum()
6         == lag
7     )
```

La situación es distinta para un rezago aplicado a una variable que ya contenía valores faltantes.

Por ejemplo, `Vol_30` tenía 29 valores iniciales sin definir. Después de aplicar `shift(1)`, la nueva columna `Vol_30_Lag_1` tendrá 30 valores faltantes iniciales.

15. Medias móviles de los rendimientos

Una media móvil de tamaño w se define como

$$M_{t,w} = \frac{1}{w} \sum_{i=t-w+1}^{t} r_i.$$

Se construirán medias móviles de 5, 10 y 20 días:

```
ventanas_ma = [5, 10, 20]

for w in ventanas_ma:
    datos[f"Mean_Return_{w}"] = (
        datos["Return"]
        .rolling(window=w)
        .mean()
    )
```

Por ejemplo,

$$\texttt{Mean_Return_5}_t = \frac{r_t + r_{t-1} + r_{t-2} + r_{t-3} + r_{t-4}}{5}.$$

La media móvil proporciona información sobre la dirección promedio de los rendimientos durante un periodo reciente.

16. Verificación manual de una media móvil

Seleccionamos una observación intermedia y reproducimos manualmente una media de 20 días:

```
w = 20
i = 100

ventana_manual = (
    datos["Return"]
    .iloc[i-w+1:i+1]
)

media_manual = ventana_manual.mean()
media_python = datos["Mean_Return_20"].iloc[i]

print("Manual:", media_manual)
```

```
print("Rolling:", media_python)

assert len(ventana_manual) == w
assert np.isclose(
    media_manual,
    media_python
)
```

Esta prueba confirma que la media correspondiente a la fecha t utiliza observaciones que terminan precisamente en esa fecha.

17. Media móvil del rendimiento absoluto

Otra medida sencilla consiste en calcular la magnitud promedio de los movimientos recientes:

$$A_{t,w} = \frac{1}{w} \sum_{i=t-w+1}^{t} |r_i|.$$

En Python:

```
for w in ventanas_ma:
    datos[f"Mean_Abs_Return_{w}"] = (
        datos["Abs_Return"]
        .rolling(window=w)
        .mean()
    )
```

Se obtendrán

`Mean_Abs_Return_5`, `Mean_Abs_Return_10`, `Mean_Abs_Return_20`.

Esta medida guarda relación con la volatilidad, pero no coincide con ella. La volatilidad mide dispersión alrededor de la media local; la media de $|r_t|$ resume directamente la magnitud promedio de los movimientos.

18. Comparación entre volatilidad y movimiento absoluto promedio

Una comparación útil consiste en visualizar

`Mean_Abs_Return_20`

junto con

`Vol_20`.

En Python:

```
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    datos.index,
    datos["Mean_Abs_Return_20"],
    label="Media de |Return|, 20 dias",
    linewidth=1
)

ax.plot(
    datos.index,
    datos["Vol_20"],
    label="Volatilidad 20 dias",
    linewidth=1
)

ax.set_title(
    "Magnitud promedio y volatilidad historica"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Magnitud")
ax.legend()

plt.show()
```

Ambas series pueden moverse de forma similar durante episodios turbulentos, pero no deben interpretarse como la misma medida.

19. Indicador de momentum

Un indicador simple de tendencia reciente puede construirse comparando el precio ajustado actual con el observado k sesiones atrás.

Definimos

$$M_{t,k} = \log\left(\frac{P_t}{P_{t-k}}\right).$$

Se utilizarán horizontes de 5 y 20 días:

```
datos["Momentum_5"] = np.log(
    datos["Adj Close"] /
    datos["Adj Close"].shift(5)
)

```

```
datos["Momentum_20"] = np.log(
    datos["Adj Close"] /
    datos["Adj Close"].shift(20)
)
```

Si

$$M_{t,k} > 0,$$

el precio actual es mayor que el observado k sesiones antes.

Si

$$M_{t,k} < 0,$$

el precio actual es menor.

> **Interpretación.** El momentum no pretende predecir por sí mismo el comportamiento futuro. En este proyecto se utiliza únicamente como una variable descriptiva de la tendencia reciente que posteriormente podrá incorporarse a los modelos.

20. Verificación del momentum

Seleccionamos una posición intermedia:

```
i = 100

momentum_manual = np.log(
    datos["Adj Close"].iloc[i] /
    datos["Adj Close"].iloc[i - 20]
)

momentum_python = (
    datos["Momentum_20"].iloc[i]
)

print("Manual:", momentum_manual)
print("Python:", momentum_python)

assert np.isclose(
    momentum_manual,
    momentum_python
)
```

21. Cambio relativo en el volumen

El volumen de operación puede cambiar considerablemente entre sesiones. Una forma sencilla de describir ese cambio es

$$\Delta V_t = \frac{V_t}{V_{t-1}} - 1.$$

En Python:

```
datos["Volume_Change"] = (
    datos["Volume"]
    .pct_change(fill_method=None)
)
```

Un valor positivo indica que el volumen aumentó respecto a la sesión anterior; un valor negativo indica una disminución.

Por ejemplo,

$$\Delta V_t = 0.25$$

representa un incremento aproximado de 25 % respecto al día anterior.

22. Rango intradiario

Los precios máximo y mínimo de una sesión proporcionan información adicional sobre la amplitud de movimientos observados durante el día.

Se definirá

$$R_t = \frac{H_t - L_t}{C_t},$$

donde H_t, L_t y C_t representan el precio máximo, mínimo y cierre del día t.

En Python:

```
datos["Range_HL"] = (
    (datos["High"] - datos["Low"]) /
    datos["Close"]
)
```

Por construcción,

$$R_t \geq 0,$$

siempre que

$$H_t \geq L_t$$

y el precio de cierre sea positivo.

Podemos verificarlo mediante

```
assert (
    datos["High"] >= datos["Low"]
).all()

assert (
    datos["Close"] > 0
).all()

assert (
    datos["Range_HL"] >= 0
).all()
```

23. Relación entre volatilidad de corto y largo plazo

En la Semana 6 se introdujo de manera exploratoria el cociente

$$Q_t = \frac{\sigma_{t,5}}{\sigma_{t,30}}.$$

Ahora se incorporará formalmente como variable predictora:

```
datos["Ratio_Vol_5_30"] = np.where(
    datos["Vol_30"] > 0,
    datos["Vol_5"] / datos["Vol_30"],
    np.nan
)
```

Cuando

$$Q_t > 1,$$

la volatilidad reciente de cinco días es mayor que la volatilidad calculada con treinta días.

Cuando

$$Q_t < 1,$$

la volatilidad de corto plazo es menor que la medida de horizonte más largo.

Esta variable resume en una sola cantidad la comparación entre dos escalas temporales.

24. Visualización del cociente de volatilidades

Podemos visualizar Q_t mediante

```
fig, ax = plt.subplots(figsize=(12, 4))

ax.plot(
    datos.index,
    datos["Ratio_Vol_5_30"],
    linewidth=0.9
)

ax.axhline(
    1,
    linestyle="--",
    linewidth=1
)

ax.set_title(
    "Relacion entre volatilidad de corto y largo plazo"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Vol_5 / Vol_30")

plt.show()
```

La línea horizontal en 1 facilita la identificación de periodos en los cuales la volatilidad de corto plazo supera a la de largo plazo.

25. Resumen de variables construidas

Las nuevas variables de la Semana 7 pueden organizarse de la siguiente forma:

Grupo	Variables
Rezagos de rendimiento	`Return_Lag_1`, `Return_Lag_2`, `Return_Lag_5`, `Return_Lag_10`
Rezagos de magnitud	`Abs_Return_Lag_1`, `Abs_Return_Lag_2`, `Abs_Return_Lag_5`, `Abs_Return_Lag_10`
Rezagos cuadrados	`Sq_Return_Lag_1`, `Sq_Return_Lag_2`, `Sq_Return_Lag_5`, `Sq_Return_Lag_10`
Rezagos de volatilidad	`Vol_5_Lag_1`, `Vol_10_Lag_1`, `Vol_20_Lag_1`, `Vol_30_Lag_1`
Medias móviles	`Mean_Return_5`, `Mean_Return_10`, `Mean_Return_20`
Magnitud promedio	`Mean_Abs_Return_5`, `Mean_Abs_Return_10`, `Mean_Abs_Return_20`
Momentum	`Momentum_5`, `Momentum_20`
Otros indicadores	`Volume_Change`, `Range_HL`, `Ratio_Vol_5_30`

26. Inspección de las nuevas variables

Una primera revisión puede realizarse con

```
columnas_nuevas = [
    col for col in datos.columns
    if col not in datos_vol.columns
]

print("Numero de variables nuevas:")
print(len(columnas_nuevas))

print("\nVariables nuevas:")
print(columnas_nuevas)
```

También conviene inspeccionar una muestra:

```
datos[
    [
        "Return",
        "Return_Lag_1",
        "Return_Lag_5",
        "Mean_Return_5",
        "Mean_Abs_Return_20",
        "Vol_20",
        "Vol_20_Lag_1",
        "Momentum_20",
        "Volume_Change",
        "Range_HL",
        "Ratio_Vol_5_30"
    ]
].head(35)
```

27. Estadística descriptiva

Podemos obtener un resumen de algunas variables mediante

```
variables_resumen = [
    "Return_Lag_1",
    "Return_Lag_5",
    "Mean_Return_5",
    "Mean_Return_20",
    "Mean_Abs_Return_20",
    "Momentum_5",
    "Momentum_20",
    "Volume_Change",
    "Range_HL",
    "Ratio_Vol_5_30"
]

resumen = (
    datos[variables_resumen]
    .describe()
    .T
)

resumen
```

La tabla permite revisar magnitudes, posibles valores extremos y diferencias de escala entre predictores.

28. Matriz de correlación exploratoria

Como análisis exploratorio puede calcularse la correlación entre algunas variables:

```
variables_corr = [
    "Return",
    "Return_Lag_1",
    "Return_Lag_5",
    "Abs_Return",
    "Mean_Abs_Return_20",
    "Vol_5",
    "Vol_20",
    "Vol_30",
    "Momentum_20",
    "Range_HL",
    "Ratio_Vol_5_30"
]

correlaciones = (
    datos[variables_corr]
    .corr()
)
```

```

correlaciones
```

Una correlación alta entre predictores no implica un error. Muchas variables están construidas a partir de la misma serie y, por tanto, pueden contener información parcialmente redundante.

La selección formal de variables se realizará más adelante, cuando se implementen los modelos predictivos.

29. Valores faltantes en la base de predictores

Las nuevas transformaciones generan valores faltantes estructurales al comienzo de la serie.

Por ejemplo:

- `Return_Lag_10` necesita 10 observaciones previas;
- `Mean_Return_20` necesita una ventana completa de 20 observaciones;
- `Momentum_20` necesita conocer el precio de 20 sesiones antes;
- `Vol_30_Lag_1` hereda los 29 valores faltantes de `Vol_30` y añade un desplazamiento adicional.

Podemos estudiar el patrón mediante

```
nan_por_columna = (
    datos.isna()
    .sum()
    .sort_values(ascending=False)
)

nan_por_columna.head(20)
```

> **Decisión del proyecto.** En esta semana no se eliminarán todavía las observaciones iniciales con valores faltantes. Primero se construirá la variable objetivo en la Semana 8 y después se determinará la muestra final disponible para modelado.

30. Validaciones automáticas adicionales

Conviene convertir algunas propiedades en pruebas automáticas.

30.1. Rezagos correctos

```
i = 100

for lag in lags:
```

```
    esperado = datos["Return"].iloc[i - lag]
    obtenido = datos[f"Return_Lag_{lag}"].iloc[i]

    assert np.isclose(
        esperado,
        obtenido
    )
```

30.2. Rango intradiario no negativo

```
valores = datos["Range_HL"].dropna()

assert (valores >= 0).all()
assert np.isfinite(valores).all()
```

30.3. Cociente de volatilidades finito

```
valores = (
    datos["Ratio_Vol_5_30"]
    .dropna()
)

assert np.isfinite(valores).all()
```

30.4. No utilizar desplazamientos futuros

La revisión del notebook debe confirmar que ninguna variable explicativa fue construida mediante

$$\texttt{shift(-1)}, \texttt{shift(-2)}, \ldots$$

31. ¿Debe estandarizarse la base en esta semana?

Todavía no.

La estandarización de una variable,

$$z_t = \frac{x_t - \mu}{s},$$

requiere estimar una media μ y una desviación estándar s.

Cuando posteriormente se divida la serie en entrenamiento, validación y prueba, estos parámetros deberán calcularse utilizando únicamente el conjunto de entrenamiento.

Por ello, en esta semana no se aplicará `StandardScaler` ni otra transformación que utilice información global de toda la serie.

> **Prevención de fuga de información.** Calcular parámetros de transformación utilizando toda la base antes de separar entrenamiento y prueba puede introducir información indirecta del futuro. Por esta razón, cualquier escalamiento se realizará dentro del pipeline de modelado en semanas posteriores.

32. Construcción de la base final de la semana

La base completa conserva las variables originales y añade todos los predictores construidos durante la semana.

Podemos realizar una copia final mediante

```
datos_features = datos.copy()
```

Antes de exportar se verificará que la base conserve el mismo número de filas que la base de entrada:

```
assert (
    datos_features.shape[0]
    == datos_vol.shape[0]
)

assert (
    datos_features.index
    .equals(datos_vol.index)
)
```

Estas pruebas garantizan que la construcción de variables no alteró el calendario de observaciones.

33. Exportación

La base se almacenará mediante

```
datos_features.to_csv(
    archivo_features,
    index=True
)
```

El pipeline queda ahora

$$\texttt{raw} \longrightarrow \texttt{clean} \longrightarrow \texttt{returns} \longrightarrow \texttt{volatility} \longrightarrow \texttt{features}.$$

34. Verificación del archivo exportado

Después de guardar la base, conviene leerla nuevamente:

```
verificacion = pd.read_csv(
    archivo_features,
    index_col=0,
    parse_dates=True
)
```

Se verifican dimensiones, orden temporal y unicidad:

```
assert (
    verificacion.shape
    == datos_features.shape
)

assert (
    verificacion.index
    .is_monotonic_increasing
)

assert verificacion.index.is_unique
```

También se comprueba la presencia de algunas variables clave:

```
columnas_clave = [
    "Return_Lag_1",
    "Return_Lag_10",
    "Vol_20_Lag_1",
    "Mean_Return_20",
    "Mean_Abs_Return_20",
    "Momentum_20",
    "Volume_Change",
    "Range_HL",
    "Ratio_Vol_5_30"
]

for col in columnas_clave:
    assert col in verificacion.columns
```

35. Ejercicios

1. Construya los rezagos 1, 2, 5 y 10 de `Return`, `Abs_Return` y `Sq_Return`. Seleccione una fecha intermedia y verifique manualmente al menos dos de los valores obtenidos.
2. Construya las medias móviles de 5, 10 y 20 días de `Return` y `Abs_Return`. Compare `Mean_Abs_Return_20` con `Vol_20` y explique por qué las dos medidas no son iguales.
3. Calcule `Momentum_5` y `Momentum_20`. Identifique las cinco fechas con mayor y las cinco fechas con menor `Momentum_20`. Interprete el signo del indicador.

4. Construya `Volume_Change` y `Range_HL`. Obtenga estadísticas descriptivas de ambas variables. Determine si las fechas con mayor rango intradiario tienden a coincidir con periodos de elevada `Vol_20`.
5. Construya `Ratio_Vol_5_30`. Calcule qué proporción de observaciones válidas satisface

   $$\texttt{Ratio_Vol_5_30} > 1.$$

   Identifique algunos periodos en los cuales el cociente alcance valores particularmente altos.
6. Explique con sus propias palabras por qué una variable construida mediante `shift(1)` puede utilizarse como predictor, mientras que una variable construida mediante `shift(-1)` no debe utilizarse como variable explicativa.

36. Checklist de cierre

Antes de considerar terminada la Semana 7, debe verificarse que:

- la base de entrada es `SPY_2015_2025_volatility.csv`;
- el índice temporal permanece ordenado y sin duplicados;
- se construyeron rezagos 1, 2, 5 y 10 de los rendimientos;
- se construyeron rezagos de `Abs_Return` y `Sq_Return`;
- se construyeron rezagos de un día de las cuatro volatilidades históricas;
- se construyeron medias móviles de rendimientos de 5, 10 y 20 días;
- se construyeron medias móviles de `Abs_Return`;
- se construyeron `Momentum_5` y `Momentum_20`;
- se construyó `Volume_Change`;
- se construyó `Range_HL`;
- se incorporó `Ratio_Vol_5_30`;
- se verificaron manualmente algunos rezagos;
- se verificó manualmente al menos una media móvil;
- se verificó manualmente el cálculo de momentum;
- se identificaron correctamente los valores faltantes estructurales;
- no se eliminaron todavía los `NaN` iniciales;
- no se utilizaron desplazamientos negativos para construir predictores;
- no se realizó todavía estandarización global de variables;
- no se construyó todavía la variable objetivo futura;
- el archivo `SPY_2015_2025_features.csv` fue exportado y verificado.

37. Producto esperado

Al finalizar la semana se deberán tener los siguientes productos:

1. notebook `07_variables_predictoras.ipynb`;
2. base `data/processed/SPY_2015_2025_features.csv`;
3. conjunto documentado de variables rezagadas;
4. medias móviles de rendimientos y rendimientos absolutos;
5. indicadores simples de momentum, volumen y rango intradiario;
6. cociente entre volatilidad de corto y largo plazo;
7. verificaciones manuales y automáticas de las transformaciones;
8. estadísticas descriptivas de las nuevas variables;
9. matriz exploratoria de correlaciones.

Conexión con la Semana 8. La base construida en esta semana contiene exclusivamente variables conocidas hasta cada fecha. En la Semana 8 se definirá la volatilidad futura a 5, 10 o 20 días que funcionará como variable objetivo. A partir de ese momento se distinguirán formalmente las variables explicativas X_t de la respuesta futura Y_t.